

⇒ Lexical Analyser

• Jobs:

- 1) Spell checking
- 2) Remove Comments
- 3) Remove Whitespace
- 4) Define line number
- 5) Convert to lexeme and token to store in symbol table.

eg:

int a = 10

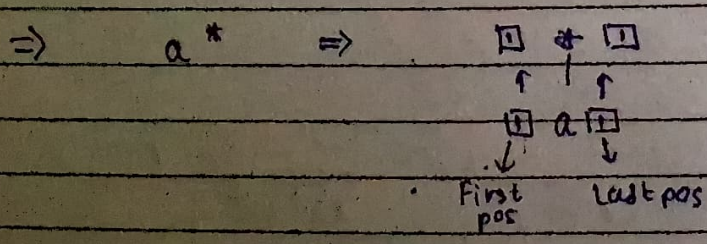
Name	Type	Datatype	Scope	Memlocation
a	identifier	int	global	1001

Symbol table

lexeme	token
datatype	int
var	a
op	=
value	10
S.op	;

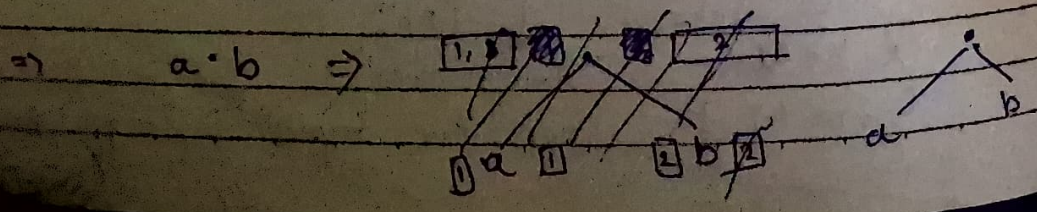
• R.F. to D.F.A.

First pos, Last pos, Follow pos and Nullable



Nullable = Yes

F.P. = Same as the children
L.P. =



Next node is *
~~a is a leaf node~~

if (a is ^{not} nullable)

First pos $\Rightarrow fa \cup fb$

else

First pos $\Rightarrow fa$

if (b is not nullable)

Last pos $\Rightarrow la \cup lb$

else

Last pos $\Rightarrow lb$

Nullable = No

F.P. = a is not nullable ? ~~fa~~ : $fa \cup fb$

L.P. = b is not nullable ? ~~lb~~ : $lb \cup la$

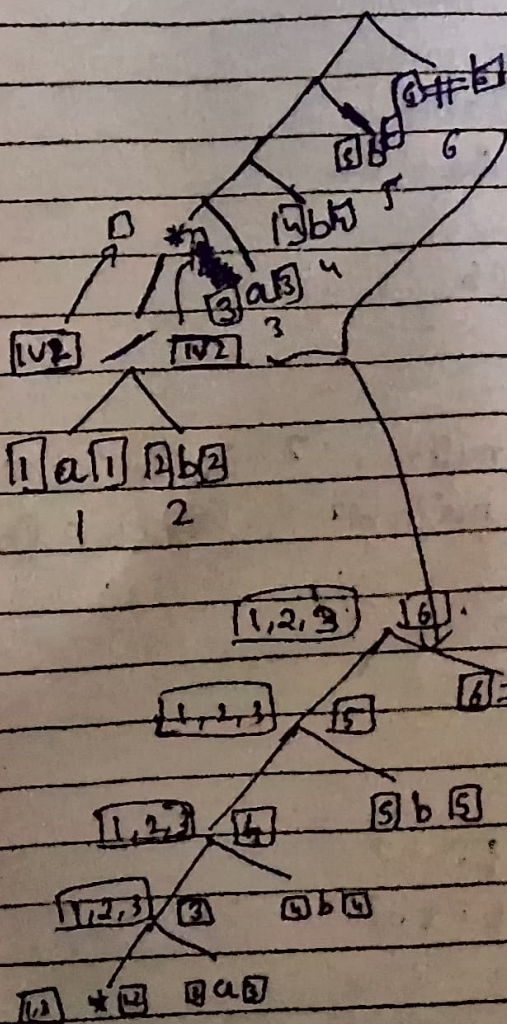
$\Rightarrow a + b$ or a / b

Nullable = No

F.P. = $FP(a) \cup FP(b)$

L.P. = $LP(a) \cup LP(b)$

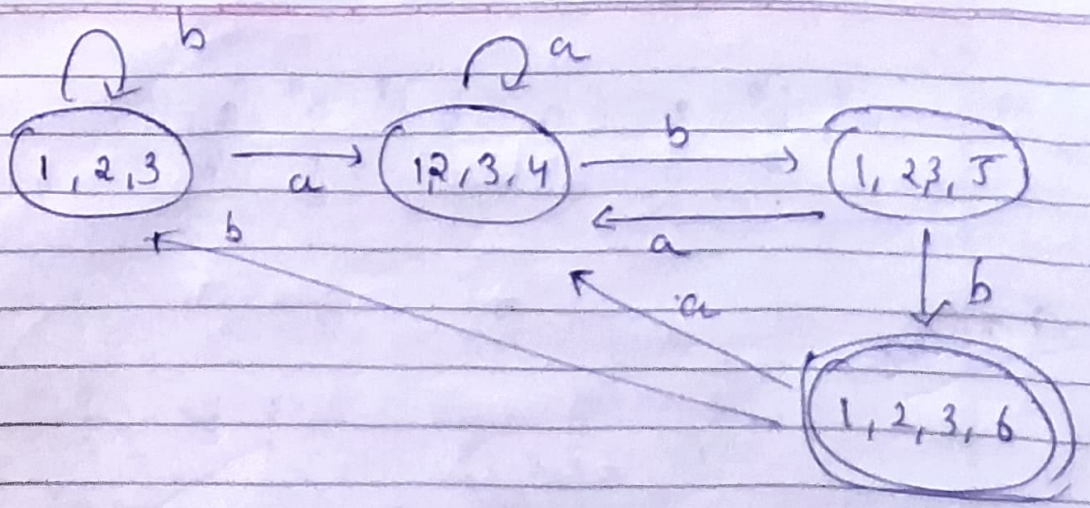
Eg: $(a+b)^* a b b \#$ ↗ end of string
 1 2 3 4 5 6



Derivation tree

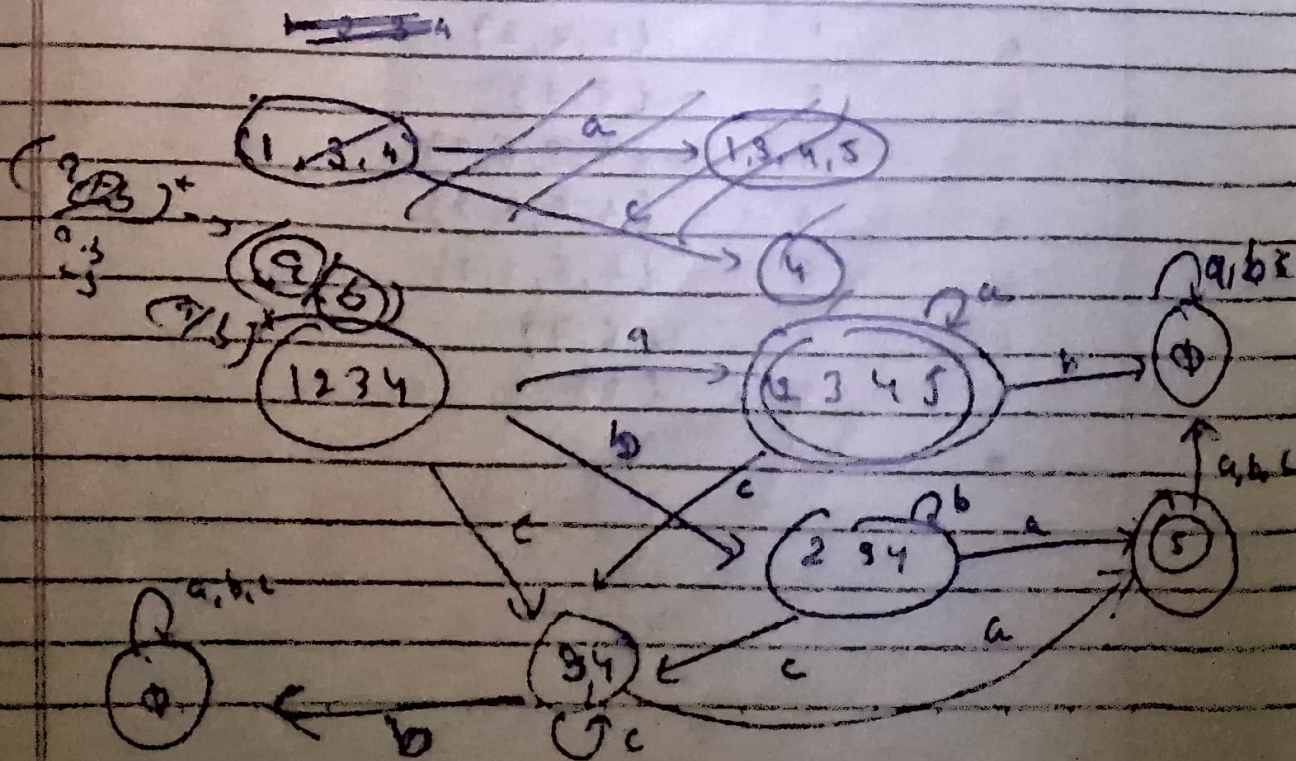
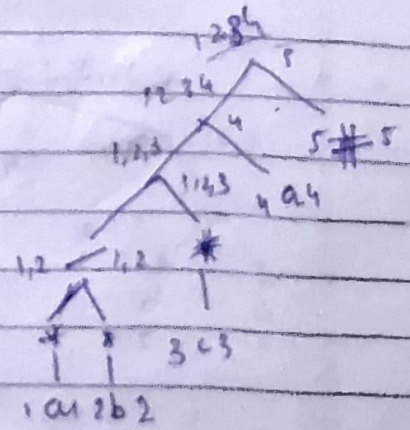
	Node	follow
a	1	{1,2,3}
b	2	{1,2,3}
a	3	{4}
b	4	{5}
b	5	{6}
#	6	-

select node
then see where
is a and b
[if two follow
position
union]



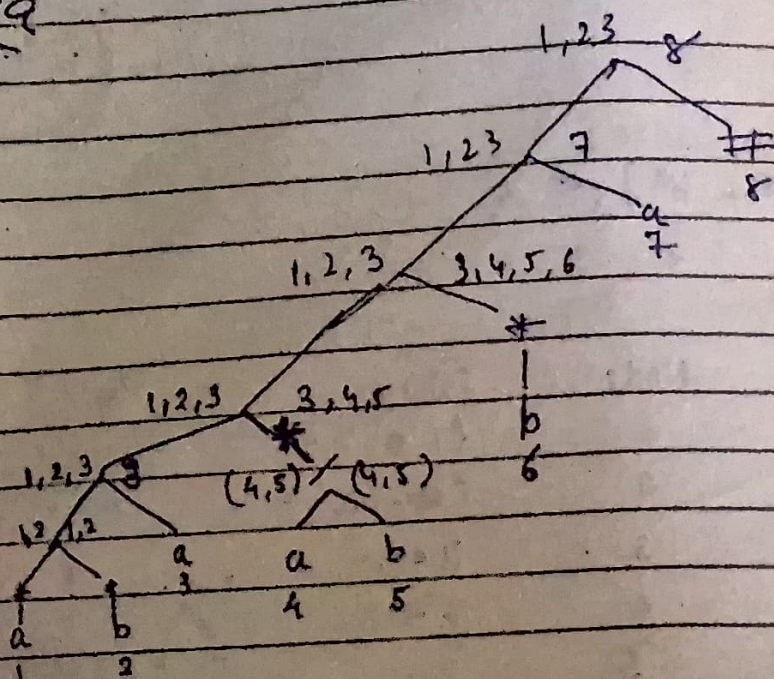
eg: $(a^* + b^*)c^*a$
 1 2 3 4

	Node	follow
a^*	1	{1, 2, 3, 4}
b^*	2	{2, 3, 4}
c^*	3	{3, 4}
a	4	{5}
$\#$	5	-



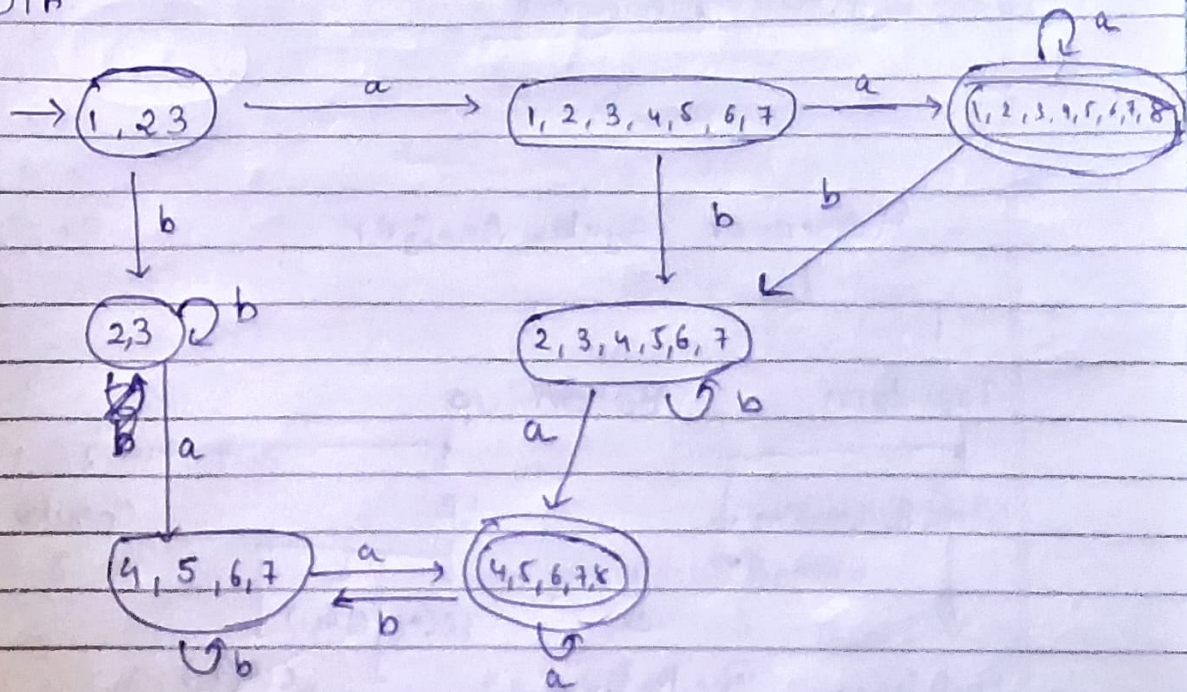
2) $a^* b^* a (a/b)^* b^* a \#$
 1 2 3 4 5 6 7 8

aaabba

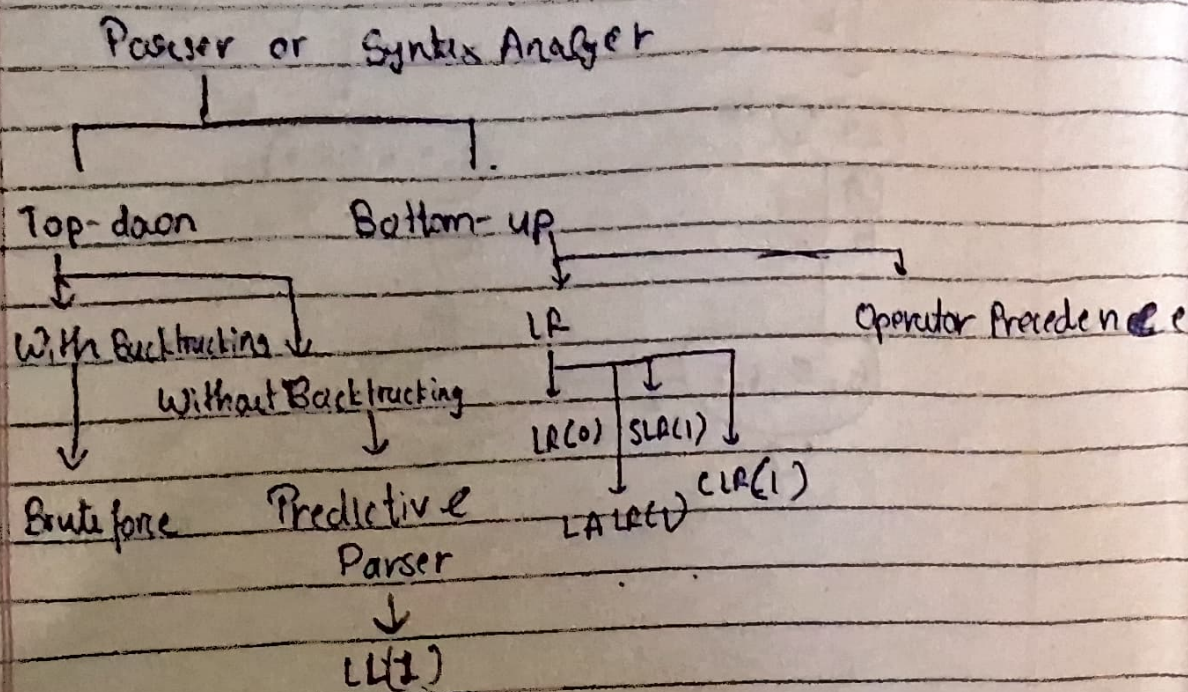


	Node	Follows
a	1	{1, 2, 3}
b	2	{2, 3}
a	3	{4, 5, 6, 7}
a	4	{4, 5, 6, 7}
b	5	{4, 5, 6, 7}
b	6	{6, 7}
a	7	{8}
#	8	-

DFA



= Syntax Analyser

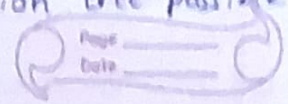


• Top-Down Parser

• For CFG,

- If the grammar is ambiguous then parser can't be created.
- If the grammar is Non-deterministic then, parser can't be created except Brute force parser, or convert in Deterministic
- If the grammar has left recursion then also parser can't be created.

Ambiguous Grammar: More than one derivation tree possible



factoring

- Left-factoring (Non-Deterministic $\rightarrow$ Deterministic)

$$S \rightarrow aA / aB$$

$$A \rightarrow a$$

$$B \rightarrow a$$

$\Rightarrow$

$$S \rightarrow aS'$$

$$S' \rightarrow A/B$$

$$A \rightarrow a$$

$$B \rightarrow a$$

Left-factored grammar

$$LR \rightarrow PR$$

$$S \rightarrow Sa / a$$

"

$\rightarrow$

$$S \rightarrow aS'$$

$$S' \rightarrow aS' / \epsilon$$

eg. $S \rightarrow Sa / Sb / c$

$$S \rightarrow SX / c$$

$$X \rightarrow a/b$$

$$S \rightarrow cS'$$

$$S' \rightarrow aS' / bS' / \epsilon$$

$$S \rightarrow \text{~~XXXXXX~~} cS'$$

$$S' \rightarrow XS' / \epsilon$$

$$X \rightarrow a/b$$

eg. $S \rightarrow Sa / Sb / x / y$

$$S \rightarrow SZ / x / y$$

$$Z \rightarrow a/b$$

$$S \rightarrow xS' / yS'$$

$$S' \rightarrow aS' / bS' / \epsilon$$

$$S \rightarrow \text{~~XXXXXXXXXX~~} xS' / yS'$$

$$S' \rightarrow zS' / \epsilon$$

$$Z \rightarrow a/b$$

eg

$$S \rightarrow Sa / b / c$$

$$S \rightarrow ~~as' / bs'~~ / es'$$

$$s' \rightarrow as' / \epsilon$$

→ Brute Force (With Backtracking)

- Brute force will work for non-deterministic grammar unlike other parser methods.

eg:

$$S \rightarrow aAB / aXYb / aB$$

$$A \rightarrow bB / bA / \epsilon$$

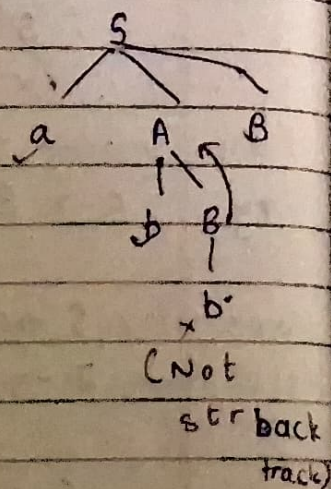
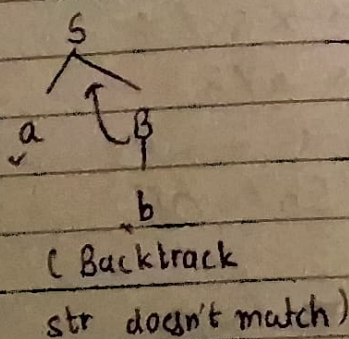
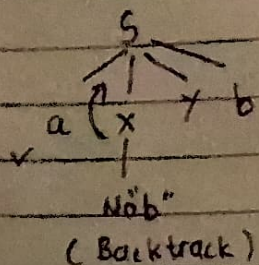
$$B \rightarrow b$$

$$X \rightarrow a$$

$$Y \rightarrow d$$

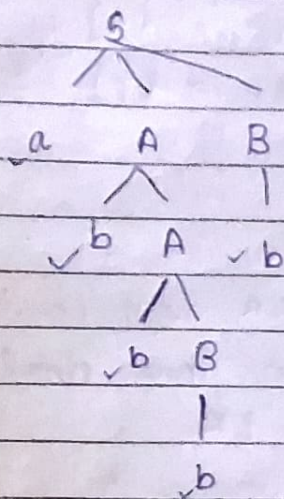
Derivation tree

$$w = abbbb$$



ε : like a pseudo terminal ; it is actual terminal if there is no other option.
And also it counted towards "first"

Page _____
Date _____



This occur after multiple backtracking. (Thus, costly)

* Without Backing → Predictive Parser → LLC1

for each production

- First : first occurring terminal_n in derivation tree

eg:-

$$\begin{aligned} S &\rightarrow aAB &\Rightarrow f(S) &= a \\ A &\rightarrow b/c &\Rightarrow f(A) &= \{b, c\} \\ B &\rightarrow d/e &\Rightarrow f(B) &= \{d, e\} \end{aligned}$$

$$\begin{aligned} S &\rightarrow AB &\Rightarrow f(S) &= \{a, \epsilon, b\} \\ A &\rightarrow a/\epsilon &\Rightarrow f(A) &= \{a, \epsilon\} \\ B &\rightarrow b/\epsilon &\Rightarrow f(B) &= \{b, \epsilon\} \end{aligned}$$

be_z A, B ⇒ ε

$$\text{If } S \rightarrow ABd \Rightarrow f(S) = \{a, \epsilon, b, d\}$$

↑
be_z A ⇒ ε

$$\text{If } S \rightarrow ABdX \Rightarrow f(S) = \{a, \epsilon, b, d\}$$

$$X \rightarrow \infty$$

eg:

$$S \rightarrow S a A / b$$

$$A \rightarrow A b / \epsilon$$

$\Downarrow$

$$\Rightarrow f(S) = \{b\}$$

$$\Rightarrow f(A) = \{ \epsilon \}$$

$$S \rightarrow b S'$$

$$S' \rightarrow a A S' / \epsilon$$

$$A \rightarrow c A'$$

$$A' \rightarrow b A / \epsilon$$

convert RR will create confusion thus simulate the tree for LR

eg:

$$S \rightarrow A a B b / B b A a \Rightarrow f(S) = \{ \epsilon, a, b \}$$

$$A \rightarrow \epsilon \Rightarrow f(A) = \{ \epsilon \}$$

$$B \rightarrow \epsilon \Rightarrow f(B) = \{ \epsilon \}$$

eg:

$$E \rightarrow E + T / T \Rightarrow f(E) = \{ \epsilon, id \}$$

$$T \rightarrow T * F / F \Rightarrow f(T) = \{ \epsilon, id \}$$

$$F \rightarrow (E) / id \Rightarrow f(F) = \{ id, \epsilon \}$$

id is together

• Follow: if $A \rightarrow \alpha A \beta$, $\beta \neq \text{null}$
 $\text{fol}(A) = \text{first}(\beta)$

if $A \rightarrow \alpha \beta$
 $\text{fol}(\beta) = \text{fol}(A)$

if $S \rightarrow \alpha A \beta$, $\beta = \text{null}$
 $\text{fol}(A) = f(\beta) \cup \text{fol}(S) - \epsilon$

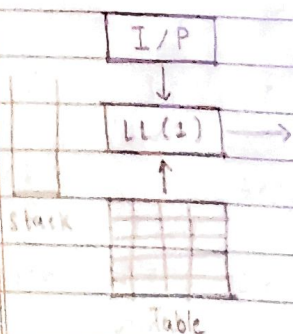
[For start production if no ~~words~~ follow can found use '\$']

eg: $S \rightarrow aSA \Rightarrow f(S) = \{a\}$
 $A \rightarrow bB \Rightarrow f(A) = \{b\}$
 $B \rightarrow \epsilon \Rightarrow f(B) = \{a\}$

$$\begin{aligned} fol(S) &= first(A) = \{a\} \\ fol(A) &= first(\epsilon) \cup fol(S) - \epsilon \\ &= fol(S) \\ &= \{a\} \\ fol(B) &= fol(A) = \{a\} \end{aligned}$$

same like

- $L(1) \rightarrow$ 1-Look ahead Symbol
 $\downarrow$ Left Recursion
 Left to Right
 Screening



eg: $E \rightarrow E + T / T$
 $T \rightarrow T * F / F$
 $F \rightarrow (E) / id$

Removing L.R. : $E \rightarrow TE' \quad E' \rightarrow +TE' / \epsilon$
 $T \rightarrow FT' \quad T' \rightarrow *FT' / \epsilon$
 $F \rightarrow (E) / id$

Creating Table:

	+	*	(	)	id	\$
E						
E'						
T						
T'						
F						

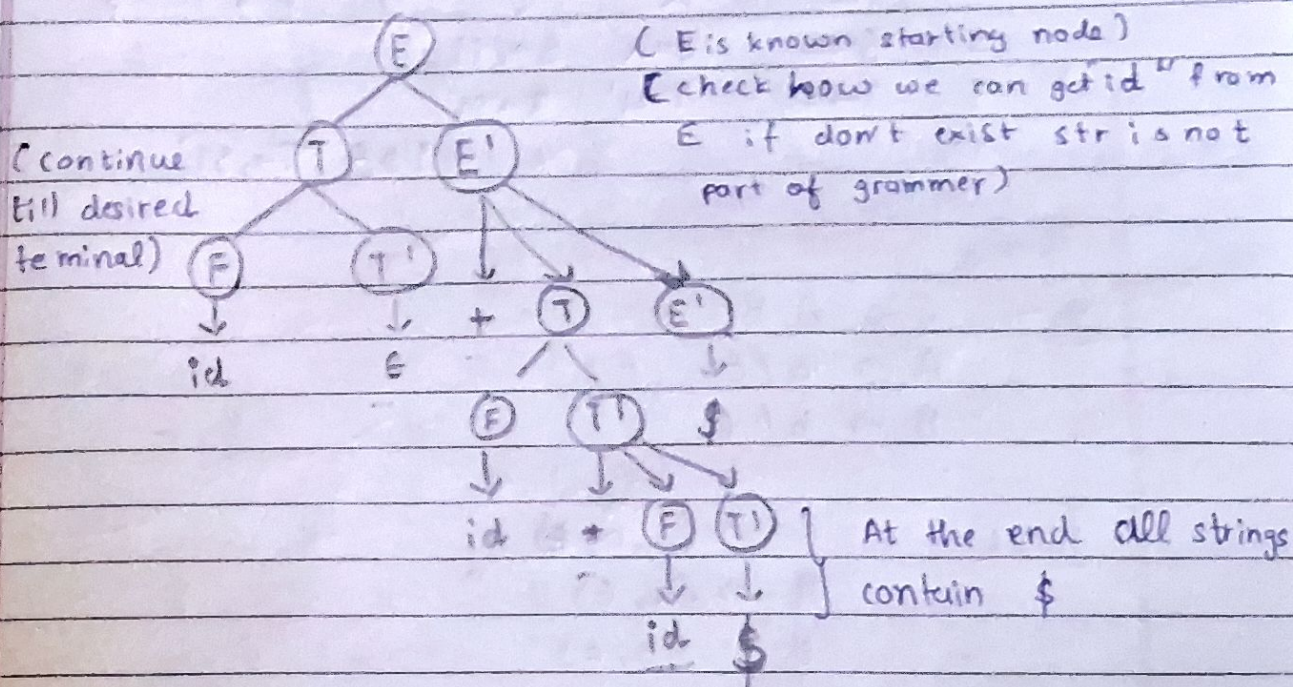
First : $E \Rightarrow \{ (, id \}$ Follow : $E \Rightarrow \{ \$,) \}$
 $E' \Rightarrow \{ +, \epsilon \}$ $E' \Rightarrow \{ \$,) \}$
 $T \Rightarrow \{ (, id \}$ $T \Rightarrow \{ +, \$,) \}$
 $T' \Rightarrow \{ *, \epsilon \}$ $T' \Rightarrow \{ +, \$,) \}$
 $F \Rightarrow \{ (, id \}$ $F \Rightarrow \{ *, +, \$,) \}$

Updating table:

	+	*	(	)	id	\$
E			$E \rightarrow TE'$		$E \rightarrow TE'$	
E'	$E' \rightarrow TE'$			$E' \rightarrow \epsilon$		$E' \rightarrow \epsilon$
T			$T \rightarrow FT'$		$T \rightarrow FT'$	
T'	$T' \rightarrow \epsilon$	$T' \rightarrow FT'$	$T' \rightarrow FT'$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F			$F \rightarrow (E)$		$F \rightarrow id$	

* If, there are more than one production in one cell then LL(1) Parser for the grammar will not exist
 $\Rightarrow$ This will generally won't occur if already checked that the grammar is deterministic and non-ambiguous

eg: I/P: id + id * id \$



eg: $S \rightarrow a / \wedge / (T)$
 $T \rightarrow T, S / \epsilon$

$\Rightarrow S \rightarrow a / \wedge / (T)$
 $T \rightarrow ST'$
 $T' \rightarrow ,ST' / \epsilon$

First: $S \Rightarrow \{a, \wedge, (\}$
 $T \Rightarrow \{a, \wedge, (\}$
 $T' \Rightarrow \{ , , \epsilon \}$

Follow: $S \Rightarrow \{ \wedge, a, (\}$
 $T \Rightarrow \{) \}$
 $T' \Rightarrow \{) \}$

Table:

	a	^	(	)	,	\$
S	$S \rightarrow a$	$S \rightarrow A$	$S \rightarrow (T)$			
T	$T \rightarrow ST'$	$T \rightarrow ST'$	$T \rightarrow ST'$			
T'				$T' \rightarrow \epsilon$	$T' \rightarrow , ST'$	

eg $S \rightarrow AB$
 $A \rightarrow a/\epsilon$
 $B \rightarrow b/\epsilon$

First : $S \Rightarrow \{a, b, \epsilon\}$
 $A \Rightarrow \{a, \epsilon\}$
 $B \Rightarrow \{b, \epsilon\}$

Follow : $A \Rightarrow \{\epsilon\}$
 $A \Rightarrow \{b, \epsilon\}$
 $B \Rightarrow \{a, \epsilon\}$

	a	b	\$
S	$S \rightarrow AB$	$S \rightarrow AB$	$S \rightarrow AB$
A	$A \rightarrow a$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
B		$B \rightarrow b$	$B \rightarrow \epsilon$

eg

$S \rightarrow aB / \epsilon$
 $B \rightarrow bC / \epsilon$
 $C \rightarrow cS / \epsilon$

First : $S \Rightarrow \{a, \epsilon\}$
 $B \Rightarrow \{b, \epsilon\}$
 $C \Rightarrow \{c, \epsilon\}$

Follow : $S \Rightarrow \{\$ \}$
 $B \Rightarrow \{\$ \}$
 $C \Rightarrow \{\$ \}$

	a	b	c	\$
S	$S \rightarrow aB$			$S \rightarrow \epsilon$
B		$B \rightarrow bC$		$B \rightarrow \epsilon$
C			$C \rightarrow cS$	$C \rightarrow \epsilon$

eg:

$S \rightarrow aSbS / bSaS / \epsilon$

First : $S \Rightarrow \{a, b, \epsilon\}$
 Follow : $S \Rightarrow \{\$, a, b\}$

Ambig

	a	b	\$
S	$S \rightarrow aSbS$	$S \rightarrow bSaS$	$S \rightarrow \epsilon$

eg: $S \rightarrow aAa / \epsilon$
 $A \rightarrow abS / \epsilon$

First : $S \Rightarrow \{a, \epsilon\}$
 $A \Rightarrow \{a, \epsilon\}$

Follow : $S \Rightarrow \{\$, a\}$
 $A \Rightarrow \{a\}$

	a	b	\$
S	$S \rightarrow aAa$ $S \rightarrow \epsilon$		$S \rightarrow \epsilon$
A			

eg: $S \rightarrow aABb$
 $A \rightarrow c / \epsilon$
 $B \rightarrow d / \epsilon$

First : $S \Rightarrow \{a\}$
 $A \Rightarrow \{c, \epsilon\}$
 $B \Rightarrow \{d, \epsilon\}$

Follow : $S \Rightarrow \{\$\}$
 $A \Rightarrow \{d, b\}$
 $B \Rightarrow \{b\}$

	a	b	c	d	\$
S	$S \rightarrow aABb$				
A		$A \rightarrow \epsilon$	$A \rightarrow c$	$A \rightarrow \epsilon$	
B		$B \rightarrow \epsilon$		$B \rightarrow d$	

eg:

$E \rightarrow E + i / i$

$\Downarrow$

$E \rightarrow i E'$

$E' \rightarrow + i E' / \epsilon$

main() {

E();

if (l == '\$') {

printf("Parsing");

}

E() {

if (l == 'i') {

match('i'),

E'(),

}

}

E'() {

if (l == '+') {

match('+');

match('i');

E'();

} else {

return;

}

}

match(char t) {

if (l == t) {

l = getChar();

else {

printf("error");

}

}

$L \ R \ (r) \rightarrow$ look ahead

Left right
moving

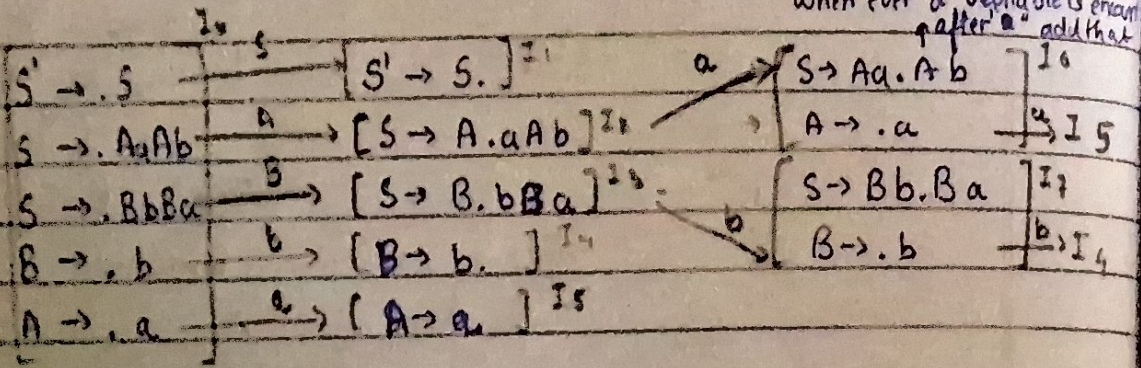
Reverse of rightmost
derivation.

eg: $S \rightarrow AaAb \text{ ①} / BbBa \text{ ②}$
 $B \rightarrow b \text{ ③}$
 $A \rightarrow a \text{ ④}$

$\rightarrow$ We augment because there may recursion S to know that it end of recursion we create this production

$S' \rightarrow .S$
 $S \rightarrow .AaAb$
 $S \rightarrow .BbBa$
 $B \rightarrow .b$
 $A \rightarrow .a$

$\hookrightarrow$ "." used for tracking in grammar.
 when ever a variable is encountered after "." add that



$[S \rightarrow Aa.Ab]^{I_6} \xrightarrow{A} [S \rightarrow AaA.b]^{I_6} \xrightarrow{b} [S \rightarrow AaAb.]^{I_6}$
 $[S \rightarrow BbBa]^{I_3} \xrightarrow{B} [S \rightarrow BbB.a]^{I_3} \xrightarrow{a} [S \rightarrow BbBa.]^{I_3}$

	Action			Go-to		
	a	b	\$	S	A	B
I ₀	S ₅	S ₄		1	2	3
I ₁			accept			
I ₂	S ₆					
I ₃		S ₇				
I ₄	(R ₃)	R ₃	R ₃			
I ₅	R ₄	R ₄	R ₄			
I ₆	S ₈			8		
I ₇		S ₄				9
I ₈		S ₁₀				
I ₉	S ₁₁					
I ₁₀	R ₁	R ₁	R ₁			
I ₁₁	R ₂	R ₂	R ₂			

reduce
3rd prod.
b → b

end
prod

⇒ str : a a . a b \$

Stack : 0 a | 5
 ↑

[I₀][a]

↓ check what is I₅ [It is R₄]

0 ~~a~~ ~~a~~ A 2 [Pop double of the total elements of Prod.]

[I₀][A]

↓ I₂

0 A 2 a 5 ~~a~~ ~~A~~

↓ After Reduction see [I₆][A]

0 ~~A~~ ~~2~~ ~~a~~ ~~5~~ ~~A~~ ~~a~~ ~~b~~ ~~\$~~

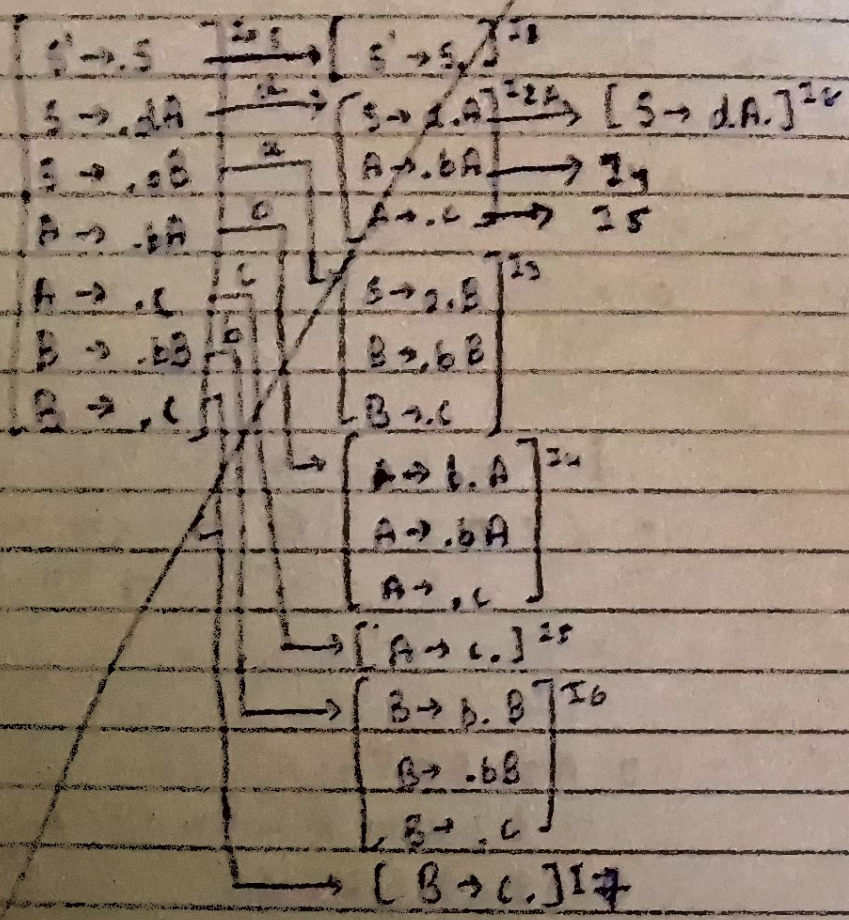
↓ [I₀][S]

0 S 1

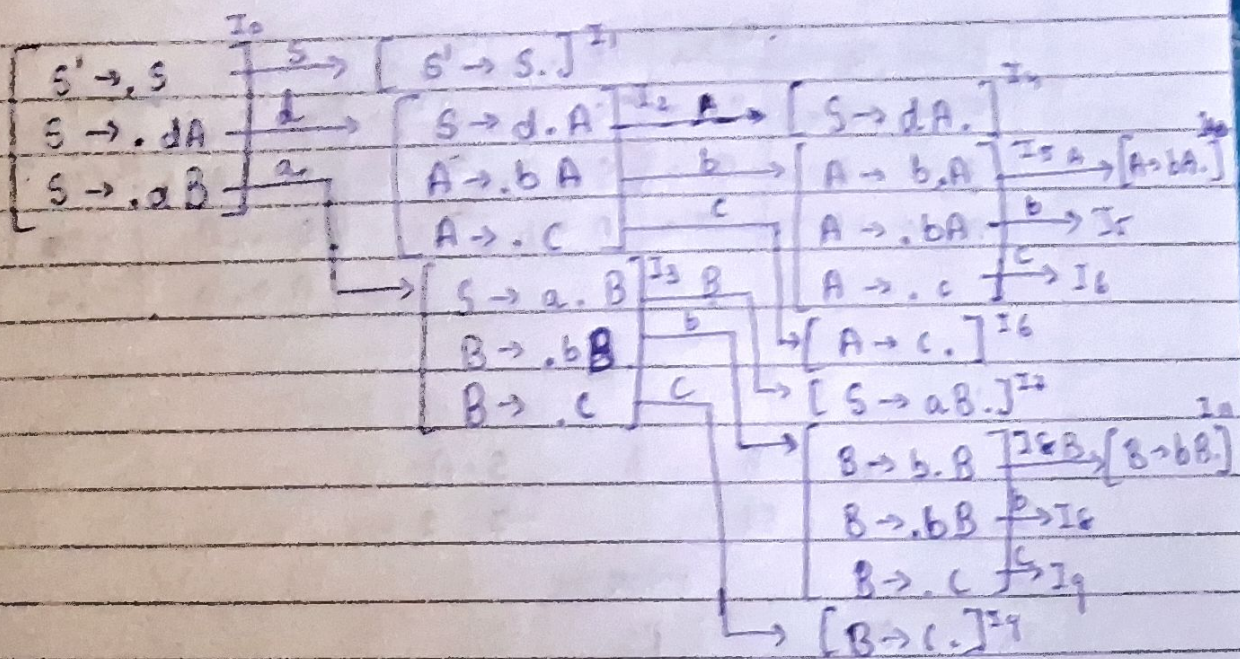
↓ Accept

$S \rightarrow dA / dB$
 $A \rightarrow bA / c$
 $B \rightarrow bB / c$

$S \rightarrow S \xrightarrow{1} [S \rightarrow S.]^{11}$
 $S \rightarrow .dA \xrightarrow{2} [S \rightarrow d.A]^{12}$
 $S \rightarrow .dB \xrightarrow{3} [S \rightarrow d.B]^{13}$
 $A \rightarrow .bA \xrightarrow{4} [A \rightarrow b.A]^{24}$
 $A \rightarrow .c \xrightarrow{5} [A \rightarrow c.]^{25}$
 $B \rightarrow .bB \xrightarrow{6} [B \rightarrow b.B]^{36}$
 $B \rightarrow .c \xrightarrow{7} [B \rightarrow c.]^{37}$



29:
 $S \rightarrow dA / aB$ ① ②
 $A \rightarrow bA / c$ ③ ④
 $B \rightarrow bB / c$ ⑤ ⑥



Actions

Go-To

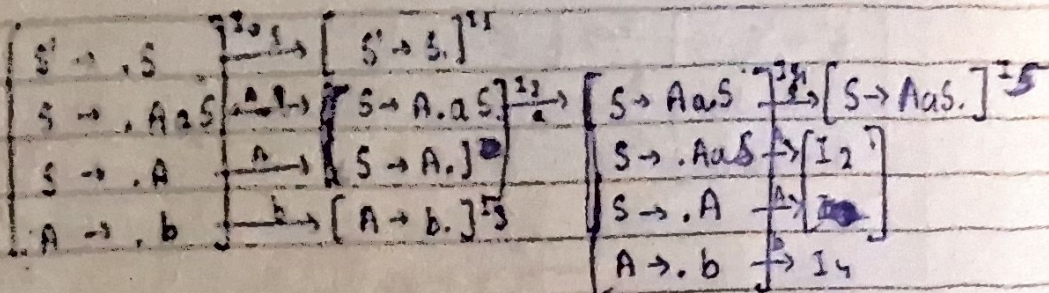
a b c d \$ S A B

I_0	S_3		S_2		1		
I_1			Accept			2	
I_2	S_5	S_6			4		
I_3	S_8	S_9					7
I_4	R_1	R_1	R_1	R_1			
I_5					10		
I_6	R_4	R_4	R_4	R_4			
I_7	R_2	R_2	R_2	R_2			
I_8							11
I_9	R_5	R_5	R_5	R_5			
I_{10}	R_3	R_3	R_3	R_3			
I_{11}	R_5	R_5	R_5	R_5			

1. LR(0)

$S \rightarrow AaS / A$

$A \rightarrow b$

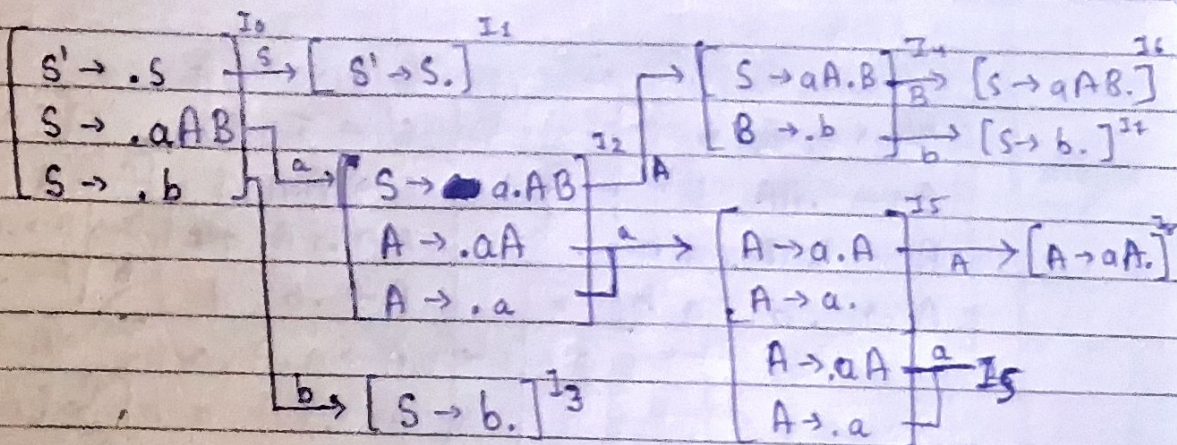


	a	b	\$	S	A
I_0		S_1		1	2
I_1					accept
I_2	S_1		R_2		
I_3	R_3		R_3		
I_4		S_3		S	
I_5			R_1		

first		follow
b	B	\$
b	B	a, \$

$S \rightarrow aAB/b$
 $A \rightarrow aA/a$
 $B \rightarrow b$

	first	follow
S	a, b	$\$$
A	a	b
B	b	$\$$



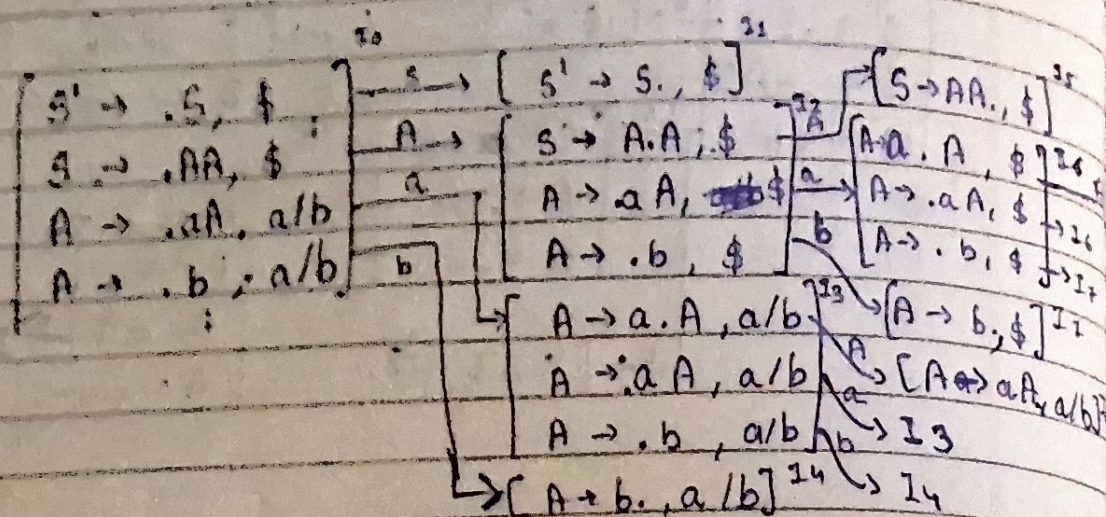
	a	b	\$	S	A	B
I_0	S_2	S_3		1		
I_1			accept			
I_2	S_5				4	
I_3			R_2			
I_4		S_7				6
I_5	S_5	R_4			8	
I_6			R_1			
I_7			R_2			
I_8		R_3				

number of blocks
 $LR(1) = LR(0) + (LR(1) \in LR(0))$

CLRCI

$S \rightarrow AA$

$A \rightarrow aA/b$



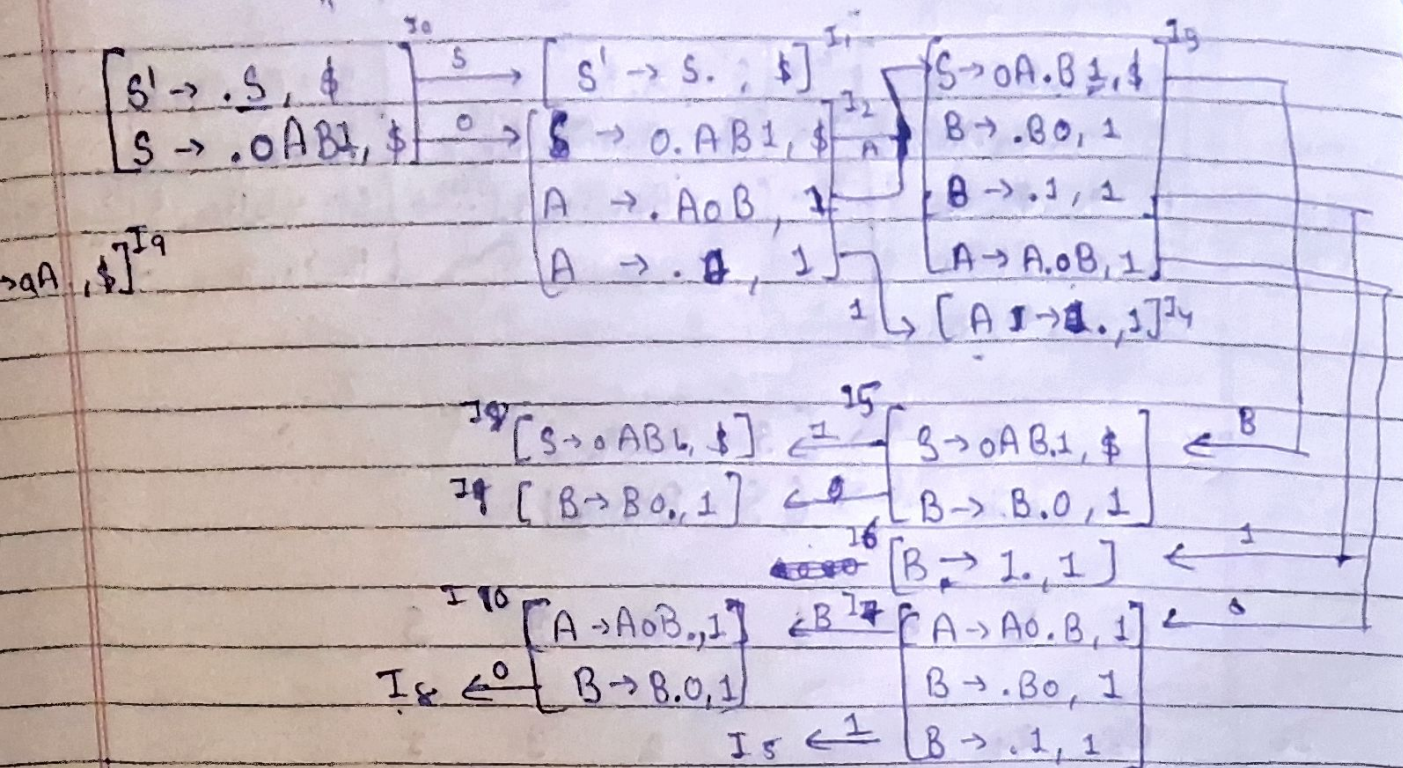
a b \$ A S

I ₀	S ₃	S ₄	D	1
I ₁			Accept	
I ₂	S ₆	S ₇		5
I ₃	S ₃	S ₄		8
I ₄	R ₃	R ₃		
I ₅			R ₁	
I ₆	S ₆	S ₇		9
I ₇			R ₃	
I ₈	R ₂	R ₂		
I ₉			R ₂	

$S \rightarrow \bullet AB1$

$A \rightarrow A \bullet B / 1$

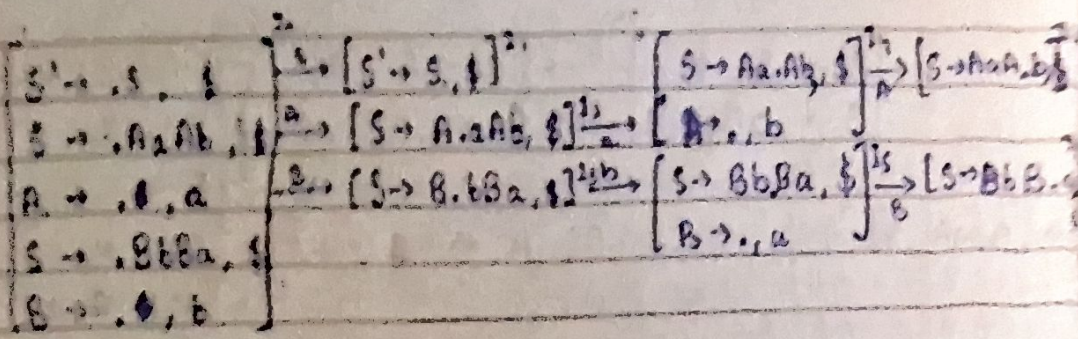
$B \rightarrow B \bullet / 1$



0 1 \$ S A B

I ₀	S ₂		1		
I ₁			accept		
I ₂		S ₄		3	
I ₃	S ₇	S ₆			5
I ₄		R ₃			
I ₅	S ₉	S ₈			
I ₆		R ₅			
I ₇		S ₅			10
I ₈			R ₁		
I ₉		R ₄			
I ₁₀	S ₈	R ₂			

$S \rightarrow A_1 A_2 / B_1 B_2$
 $A \rightarrow a$
 $B \rightarrow b$



$18 \xrightarrow{a} [S \rightarrow A_1 A_2 \cdot, \$]^{18}$
 $17 \xrightarrow{b} [S \rightarrow B_1 B_2 \cdot, \$]^{19}$

	a	b	\$	A	B	S
10	R3	R4		2	3	1
11			accept			
12	S4					
13		S5				
14		R3		6		
15	R4				7	
16	S4					
18		S9				
19			R1			
20			R2			

8) $S \rightarrow Aa / bAc / dc / bda$
 $A \rightarrow d$

$S' \rightarrow .S, \$$	$\xrightarrow{S} [S \rightarrow S., \$]^{I_1}$	
$S \rightarrow .Aa, \$$	$\xrightarrow{A} [S \rightarrow A.a, \$]^{I_2} \xrightarrow{a} [S \rightarrow Aa., \$]^{I_6}$	I_9
$S \rightarrow .bAc, \$$	$\xrightarrow{b} [S \rightarrow b.Ac, \$]^{I_3} \xrightarrow{A} [S \rightarrow bA.c, \$]^{I_6} \xrightarrow{c} [S \rightarrow bAc.]^{I_7}$	
$S \rightarrow .dc, \$$	$\xrightarrow{d} [S \rightarrow bd.a, \$]^{I_4} \xrightarrow{a} [S \rightarrow bda.]^{I_8}$	
$S \rightarrow .bda, \$$	$\xrightarrow{d} [S \rightarrow bd.a, \$]^{I_4} \xrightarrow{a} [S \rightarrow bda.]^{I_8}$	
$A \rightarrow .d, a$	$\xrightarrow{d} [A \rightarrow d., a]^{I_5} \xrightarrow{a} [A \rightarrow d., a]^{I_{10}}$	I_{10}

LALR(1)

$S \rightarrow AA$

$A \rightarrow aA/b$

Tree same as CLR(1) (See first question of CLR(1))

Table for CLR

	a	b	\$	A	S
I ₀	S ₃	S ₄		2	1
I ₁			accept		
I ₂	S ₆	S ₇		5	
I ₃	S ₃	S ₄		8	
I ₄	R ₃	R ₃			
I ₅			R ₁		
I ₆	S ₆	S ₇		9	
I ₇			R ₃		
I ₈	R ₂	R ₂			
I ₉			R ₂		

Check for same production, but different follow

up:
 $A \rightarrow b, a/b$
 $A \rightarrow b, \$$

We merge the both the states to reduce the states.

Table LA2 AC1)

	a	b	d	A	S
10	S3	S4		2	1
11			AC		
12	S6	S7		5	
136	S36	S47		98	
147	R3	R3	R3		
15			R1		
189	R2	R2	R2		

SDT

{Syntax Directed
Translation}

$$E \rightarrow E + T$$

$$E.val = E.val + T.val$$

$$E \rightarrow T$$

$$E.val = T.val$$

$$T \rightarrow T * F$$

$$T.val = T.val * F.val$$

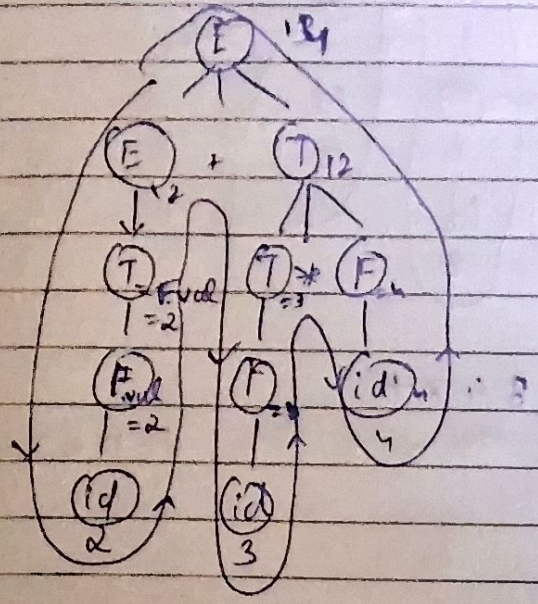
$$T \rightarrow F$$

$$T.val = F.val$$

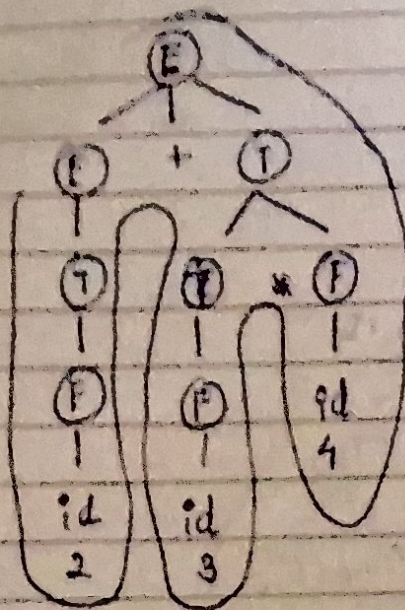
$$F \rightarrow id$$

$$F.val = num.lexval$$

2 + 3 * 4



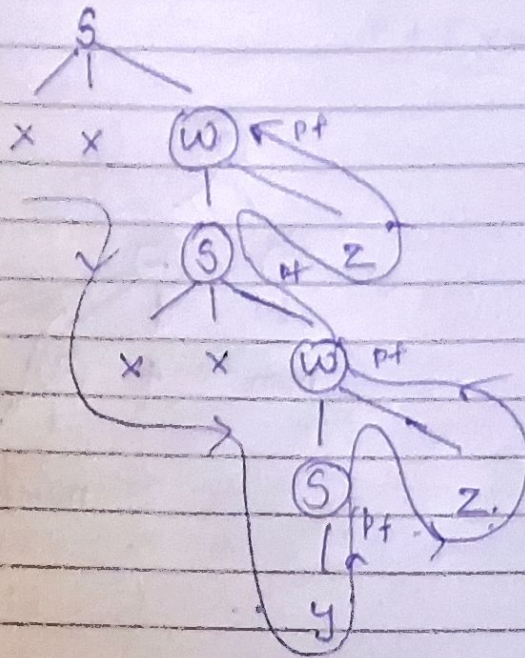
$E \rightarrow E + T$ $\{ \text{print}(' + ') \}$
 $E \rightarrow T$ $\{ \}$
 $T \rightarrow T * F$ $\{ \text{print}(' * ') \}$
 $T \rightarrow F$ $\{ \}$
 $F \rightarrow id$ $\{ \text{print}(\text{num}) \}$



Infix to Postfix : 2 3 4 * +

$S \rightarrow \alpha x w \quad \{ p.f.(1); \}$
 $\quad \quad \quad /y \quad \quad \{ p.f.(2); \}$
 $w \rightarrow Sz \quad \quad \{ p.f.(3); \}$

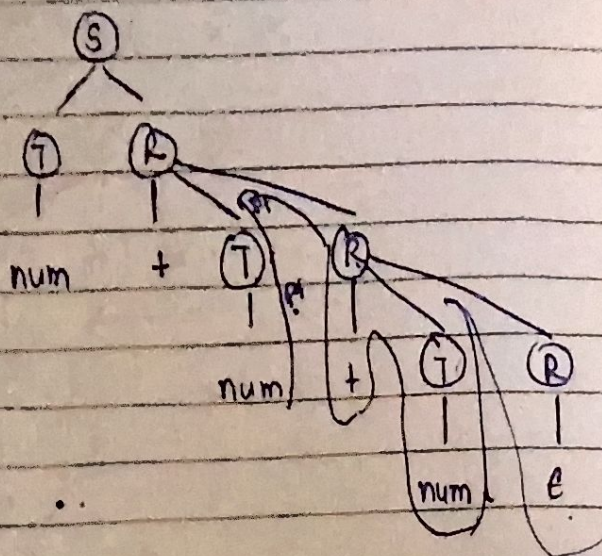
string "αxxxyzz"



Output : 2 3 1 3 1

$S \rightarrow TR$
 $R \rightarrow +T \{ \text{of } (') \} R$
 $A \rightarrow \epsilon$
 $T \rightarrow \text{num} \{ \text{pf. (number)} \}$
val

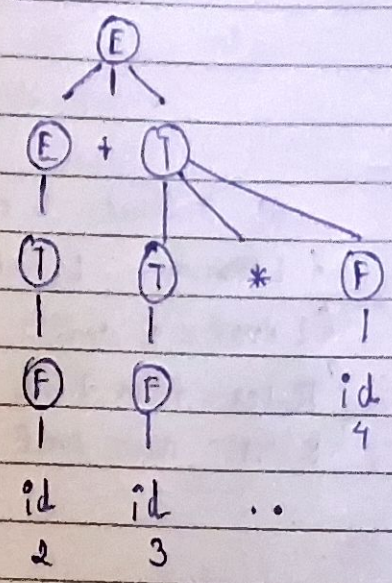
213+4



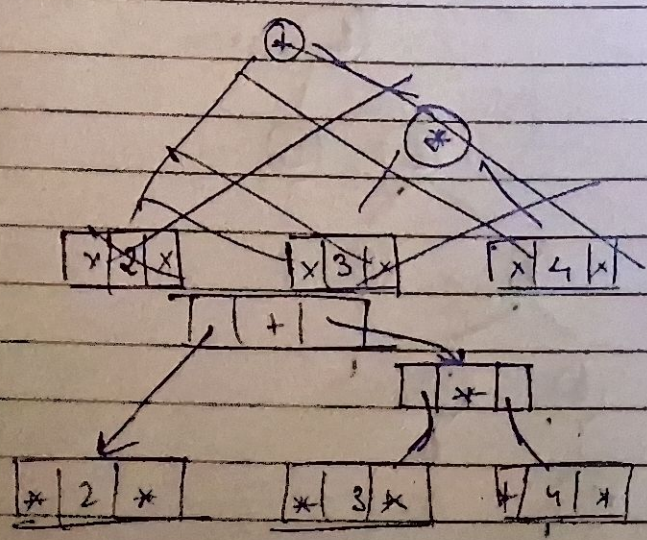
Output : ~~213+4~~ 23+4+

$E \rightarrow E + T \quad \{ E.nptr = \text{mknode}(E.nptr + T.nptr); \}$
 $\quad \quad \quad / T \quad \{ E.nptr = T.nptr \}$
 $T \rightarrow T * F \quad \{ T.nptr = \text{mknode}(T.nptr * F.nptr); \}$
 $\quad \quad \quad / F \quad \{ T.nptr = F.nptr \}$
 $F \rightarrow id. \quad \{ F.nptr = \text{mknode}(null, idnum, null); \}$

$2 + 3 * 4$



Output



$S \rightarrow L$

$L \rightarrow LB$

$L \rightarrow B$

$B \rightarrow 0$
/1

$B.val = num.dval$
 $B.dval = num$

Asked to convert ~~Decimal~~ to

Binary to decimal

1011
↓
1x2+0
2x2+1
5x2+1
11

$\Rightarrow$

$S \rightarrow L$

$\{ S.dval = L.dval \}$

$L \rightarrow LB$

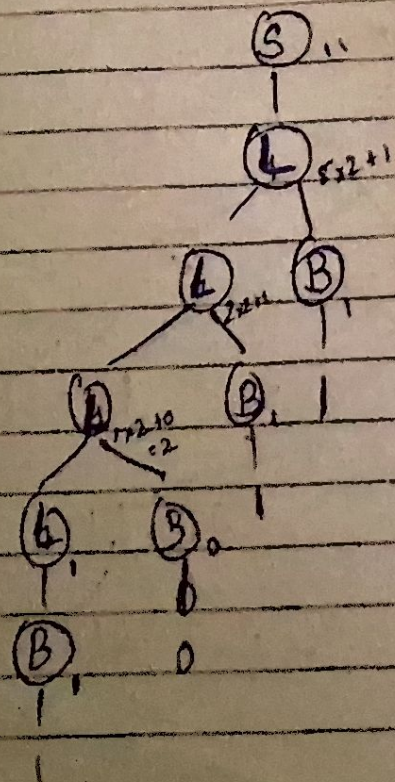
$\{ L.dval = L.dval * 2 + B.dval \}$

$L \rightarrow B$

$\{ L.dval = B.dval \}$

$B \rightarrow 0$
/1

$\{ B.dval = num.dval \}$
 $\{ B.dval = num.dval \}$



$S \rightarrow id = E$ { gen (id.num = E.place) }
 $E \rightarrow E + T$ { E.place = new temp ();
 gen (E.place = E.place + T.place) }
 $E \rightarrow T$ { E.place = T.place }
 $T \rightarrow T * F$ { T.place = new temp ();
 gen (T.place = T.place * F.place) }
 $T \rightarrow F$ { T.place = F.place }
 $F \rightarrow id$ { F.place = id.name }

$$x = a + b + c$$
